FBServiceExt Voting Business Flow

Readable operator view of the current vote path: **GET_STARTED** opens candidate selection, the next tap creates a confirmation challenge with the correct button randomized, and only the correct confirmation inside the time window creates a persisted vote and starts cooldown.

WHAT THIS DIAGRAM SHOWS

Candidate selection itself is not a vote. The business effect happens only after the confirmation challenge succeeds. This is the core anti-auto-clicker mechanism in the current flow.

CURRENT RUNTIME SHAPE

Content, timeouts, cooldown, carousel cards, and button labels are al
config-driven. Messenger messages are sent from the worker through `MetaMessengerClient`.

- Trigger / gating
- Choice / selection
- Confirmation / challenge
- Vote accepted / cooldown result
- Config driven timing / content

Stage 1: Vote Start and Candidate Carousel

Entry into the vot

STEP 1

User sends GET_STARTED

The inbound message matches one of the configured vote-start tokens.

This is the only text that opens the candidate carousel flow.

Config ref: `VotingWorkflow:VoteStartTokens`
appsettings.json: L54-L58

→

STEP 2

Worker checks token and cooldown

`ProcessMessageAsync()` ignores non-business text and stops early if the user is still in cooldown.

If cooldown is active, the user gets a friendly retry-time message instead of a carousel.

Code ref: NormalizedEventProcessor.cs
ProcessMessageAsync(): L183-L274
Cooldown gate: L228-L234

→

STEP 3

Save `OptionsSent` session

The state machine moves `Idle → VoteRequested → OptionsSent`.

The session stores the active user scope and expiry for option picking.

State refs: L243-L244
Session save + metric: L246-L264

→

STEP 4

Send candidate carousel

The worker builds a Messenger generic template from `Candidates:Items`.

Each card gets image, display name, and one postback button.

TrySendCandidateCarouselAsync(): L825-L878
MetaMessengerClient.SendGenericTemplateAsync(): L66-L90

→

RESULT

Messenger shows th carousel

One card per enabled candi using public image URLs bu from the configured base U and asset folder.

This is the first visible operator-controlled voting surface.

BuildCandidateImageUrl(): L1310
Config refs: appsettings.json L
L131

Tokens

`VotingWorkflow:VoteStartTokens` decides which incoming text starts voting.

Candidates

`Candidates:Items`, `PublicBaseUrl`, and `AssetBasePath` define the carousel content and image URLs.

Button text

`MessengerContent:CandidateVoteButtonTitleFormat` controls the on each carousel button.

Stage 2: Candidate Selection and Confirmation Challenge

The anti-auto-clic

STEP 5

User taps a candidate

STEP 6

Resolve active session and candidate

STEP 7

Create `ConfirmationPending`

STEP 8

Send one-card confirmatio challenge

The carousel button sends a postback where `payload = candidate.Id`.

This still does not count as a vote.

Source: carousel button payload

→

`ProcessPostbackAsync()` loads the active vote session and finds the candidate by payload.

If there is no active session or the candidate is unknown, the event is ignored.

ProcessPostbackAsync(): L282-L517

→

The flow moves `CandidateSelected → ConfirmationPending`.

The session now stores the selected candidate and the confirmation timeout window.

Confirmation session build + save: L369-L385

→

The worker sends a single candidate
with prompt text and **3 buttons**.

The correct candidate label is rando
across positions to reduce blind aut
clicking.

TrySendVoteConfirmationPromptAsync(): L891-L
BuildVoteConfirmationButtons(): L1194-L1235

WRONG PATH

Wrong button

If the user presses a decoy confirmation button, the session is removed and a rejected text is sent.

Wrong confirmation branch: L423-L449

TOO LATE

Expired confirmation

If the time window expires, the backend rejects the confirmation and asks the user to restar
from the candidate list.

HasSessionExpired(): L1306
HandleExpiredSessionAsync(): L767-L822

Timeout

`VotingWorkflow:ConfirmationTimeout` is currently 120 seconds.

Prompt text

`VoteConfirmationPromptFormat` builds the “confirm in N seconds” message shown above the image.

Button randomization

`VoteConfirmationCorrectButtonTitleFormat` and `VoteConfirmationDecoyButtonTitles` feed the 3-button anti-auto
clicker challenge.

Stage 3: Vote Accepted, Cooldown, and Retry Handling

This is where the business effect

STEP 9

Correct confirmation arrives

The postback payload matches the expected accept payload for the selected candidate.

Only this branch can create an accepted vote.

Accept path decision: L403-L417

→

STEP 10

`AcceptVoteAsync()`

The state machine advances through confirmation-received, persisted, notified, and finally cooldown-active.

The cooldown timestamp is computed here.

AcceptVoteAsync(): L627-L724

→

STEP 11

Persist vote and cooldown snapshot

`AcceptedVoteStore.TryAddAsync()` records the vote, and the session store keeps the cooldown window for future attempts.

Accepted vote insert: L667
Cooldown snapshot save: L670-L684

→

STEP 12

Send final success message

The user sees that the vote was accepted and when the next allowed vote time starts.

This is the visible completion of a successful vote.

TrySendVoteAcceptedMessageAsync(): L727-L741
BuildVoteAcceptedText(): L1237-L1248

→

LATER RETRY

Cooldown blocks immediate re-vote

If the user tries again du
cooldown, both message and
postback paths stop early
send the retry-time messag

Message cooldown branch: L228-L
Postback cooldown branch: L325-
Cooldown text builder: L1250-L1

Cooldown window

`VotingWorkflow:Cooldown` is currently 5 minutes and starts only after a correct confirmation.

Success text

`VoteAcceptedTextFormat` controls the final acceptance message with candidate and next allowed time.

Retry text

`CooldownActiveTextFormat` plus `CooldownTimeFormat` build the
again after HH:mm:ss” message.

What actually counts as a vote: the candidate tap from the carousel is only a selection. The business vote is created only after the confirmation challenge succeeds inside the timeout window.

Countdown nuance: Messenger generic/button templates can show the configured `120 seconds` as text, but they do not provide a native live ticking countdown bubble. Real enforcement still happens in backend session expiration and cooldown logic.